НИТУ «МИСИС»

Кафедра инженерной кибернетики

ОТЧЕТ

ПО

ЛАБОРАТОРНОЙ РАБОТЕ №1

«Методика измерения времени выполнения

заданного фрагмента программы»

учебная дисциплина «Параллельные вычисления»

Студент: Голубев И. М., БПМ-22-ПО-1

Преподаватель: Ватутин Э. И.

Оценка:

Дата защиты: 15.03.2026

2026 г.

Введение

Основная цель работы:

В соответствии с выбранным вариантом задачи измерить время её выполнения, используя выбранные способы измерения. Замер производить не менее K раз (число K задаётся пользователем). Убедиться в существовании погрешности измерения.

Описание решаемой задачи

Задача: Алгоритм быстрого преобразования Фурье (radix-2 FFT)

Входные данные: Число N – размер входной последовательности (степень двойки). Далее псевдослучайным образом генерируется входной сигнал x[0, …, N – 1] – массив комплексных чисел, число замеров K фиксировано.

Результат: Преобразованный сигнал X[0, …, N – 1] – комплексный спектр входной последовательности x[0, …, N – 1]

Способ измерения	**Аналог из методички**	**Описание**
clock_gettime(CLOCK_MONOTONIC)	GetTickCount() или Now()	Coarse OS time (<time.h>)
__rdtsc()	RDTSC	CPU cycle counter (<x86intrin.h>)
clock_gettime(CLOCK_MONOTONIC_RAW)	QueryPerformanceCounter() и QueryPerformanceFrequency()	High-res monotonic (<time.h>)

Выбранные средства для разработки

Для выполнения лабораторной работы использовались следующие программные средства и технологии:

- Язык программирования: C++17
- Библиотеки: time.h (для clock_gettime), x86intrin.h (интринсики), pthread.h
- Среда разработки: Zed
- Операционная система: Fedora Linux

Листинг кода программы

Код доступен в удалённом Github репозитории по ссылке:

https://github.com/g0lubew-iv/parallel-comp/blob/main/lab01/lab01.cpp

Ниже приведён полный листинг:

```
#include <bits/stdc++.h>
#include <time.h>
#include <unistd.h>
#include <sys/resource.h>
#include <pthread.h>
#include <x86intrin.h>

using namespace std;

static volatile double g_sink = 0.0;

static inline uint64_t rdtsc_start() {
    unsigned int eax, ebx, ecx, edx;
    eax = 0;
    __asm__ __volatile__(
        "cpuid\n\t"
        : "=a"(eax), "=b"(ebx), "=c"(ecx), "=d"(edx)
        : "a"(eax)
        : "memory"
    );
    return __rdtsc(); // RDTSC intrinsic [1]
}

static inline uint64_t rdtsc_end() {
    unsigned int aux;
    uint64_t t = __rdtscp(&aux);
    unsigned int eax, ebx, ecx, edx;
    eax = 0;
    __asm__ __volatile__(
        "cpuid\n\t"
        : "=a"(eax), "=b"(ebx), "=c"(ecx), "=d"(edx)
        : "a"(eax)
        : "memory"
    );
    return t;
}
```

```
// Method 1: monotonic OS time (ns)
static inline uint64_t nsec_now_monotonic() {
    timespec ts{};
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return uint64_t(ts.tv_sec) * 1000000000ull + uint64_t(ts.tv_nsec);
}

// Method 3: monotonic raw (ns)
static inline uint64_t nsec_now_monotonic_raw() {
    timespec ts{};
    clock_gettime(CLOCK_MONOTONIC_RAW, &ts);
    return uint64_t(ts.tv_sec) * 1000000000ull + uint64_t(ts.tv_nsec);
}

static bool pin_to_cpu0() {
    cpu_set_t cpuset;
    CPU_ZERO(&cpuset);
    CPU_SET(0, &cpuset);
    return pthread_setaffinity_np(pthread_self(), sizeof(cpu_set_t), &cpuset) == 0;
}

static void try_raise_priority() { setpriority(PRIO_PROCESS, 0, -20); }

static void fft_inplace(vector<complex<double>>& a) {
    const size_t n = a.size();

    for (size_t i = 1, j = 0; i < n; i++) {
        size_t bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }

    for (size_t len = 2; len <= n; len <<= 1) {
        const double ang = -2.0 * M_PI / double(len);
        complex<double> wlen(cos(ang), sin(ang));
        for (size_t i = 0; i < n; i += len) {
            complex<double> w(1.0, 0.0);
            for (size_t j = 0; j < len / 2; j++) {
                complex<double> u = a[i + j];
                complex<double> v = a[i + j + len / 2] * w;
                a[i + j] = u + v;
                a[i + j + len / 2] = u - v;
                w *= wlen;
            }
        }
    }
}

static double checksum(const vector<complex<double>>& a) {
```

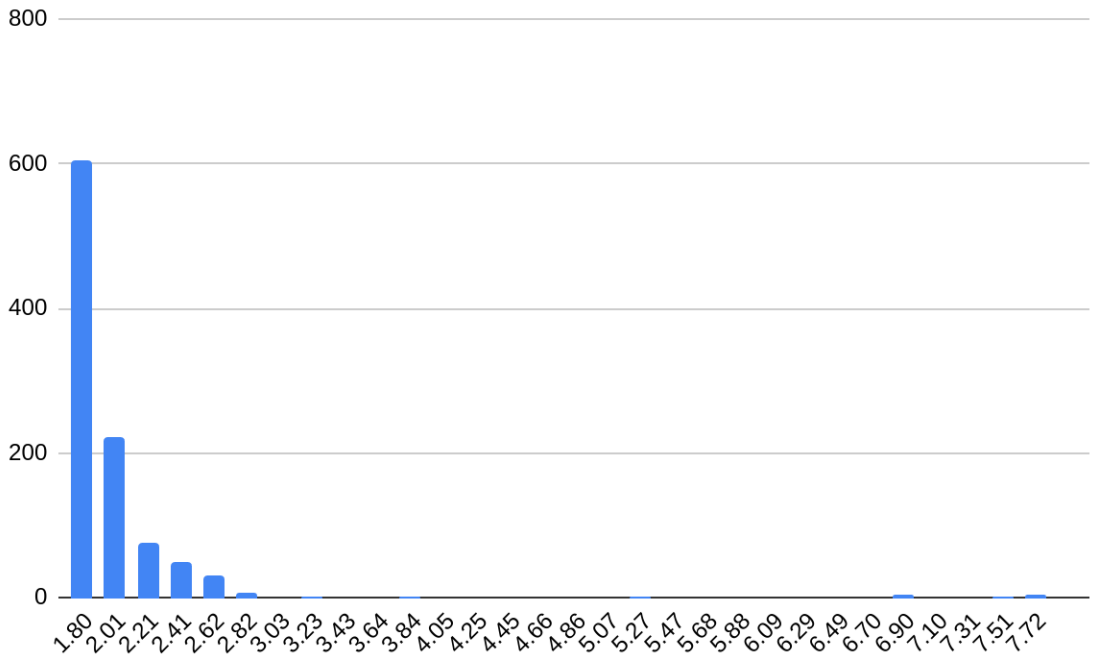
```
    double s = 0.0;
    for (auto &z : a) {
        s += z.real() * 1e-6 + z.imag() * 2e-6;
    }
    return s;
}

int main() {
    const int K = 1000;
    size_t N = 2048 * 32; // must be power of two for radix-2 FFT

    bool pinned = pin_to_cpu0();
    (void)pinned;
    try_raise_priority();

    vector<complex<double>> input(N);
    uint64_t seed = 123456789;
    auto rng = [&]() {
        seed ^= seed >> 12;
        seed ^= seed << 25;
        seed ^= seed >> 27;
        return seed * 2685821657736338717ULL;
    };

    for (size_t i = 0; i < N; i++) {
        double re = double(rng() % 1000000) / 1000000.0;
        double im = double(rng() % 1000000) / 1000000.0;
        input[i] = {re, im};
    }

    // Warm-up
    {
        auto a = input;
        fft_inplace(a);
        g_sink += checksum(a);
    }

    vector<double> t_mono_ns; t_mono_ns.reserve(K);
    vector<double> t_high_ns; t_high_ns.reserve(K);
    vector<double> t_cycles;  t_cycles.reserve(K);

    ofstream csv("res.csv");
    csv << "run,n,mono_ns,high_ns,tsc_cycles\n";

    for (int i = 0; i < K; i++) {
        // Method 1: MONOTONIC in ns
        auto a1 = input;
        uint64_t t0 = nsec_now_monotonic();
        fft_inplace(a1);
        uint64_t t1 = nsec_now_monotonic();
```

```
        g_sink += checksum(a1);
        uint64_t dmono = t1 - t0;
        t_mono_ns.push_back(double(dmono));

        // Method 2: TSC cycles/clocks
        auto a2 = input;
        uint64_t c0 = rdtsc_start();
        fft_inplace(a2);
        uint64_t c1 = rdtsc_end();
        g_sink += checksum(a2);
        uint64_t dc = c1 - c0;
        t_cycles.push_back(double(dc));

        // Method 3: high-res raw ns
        auto a3 = input;
        uint64_t ns0 = nsec_now_monotonic_raw();
        fft_inplace(a3);
        uint64_t ns1 = nsec_now_monotonic_raw();
        g_sink += checksum(a3);
        uint64_t dns = ns1 - ns0;
        t_high_ns.push_back(double(dns));

        csv << (i + 1) << "," << N << "," << dmono << "," << dns << "," << dc << "\n";
    }

    return 0;
}
```

Результаты

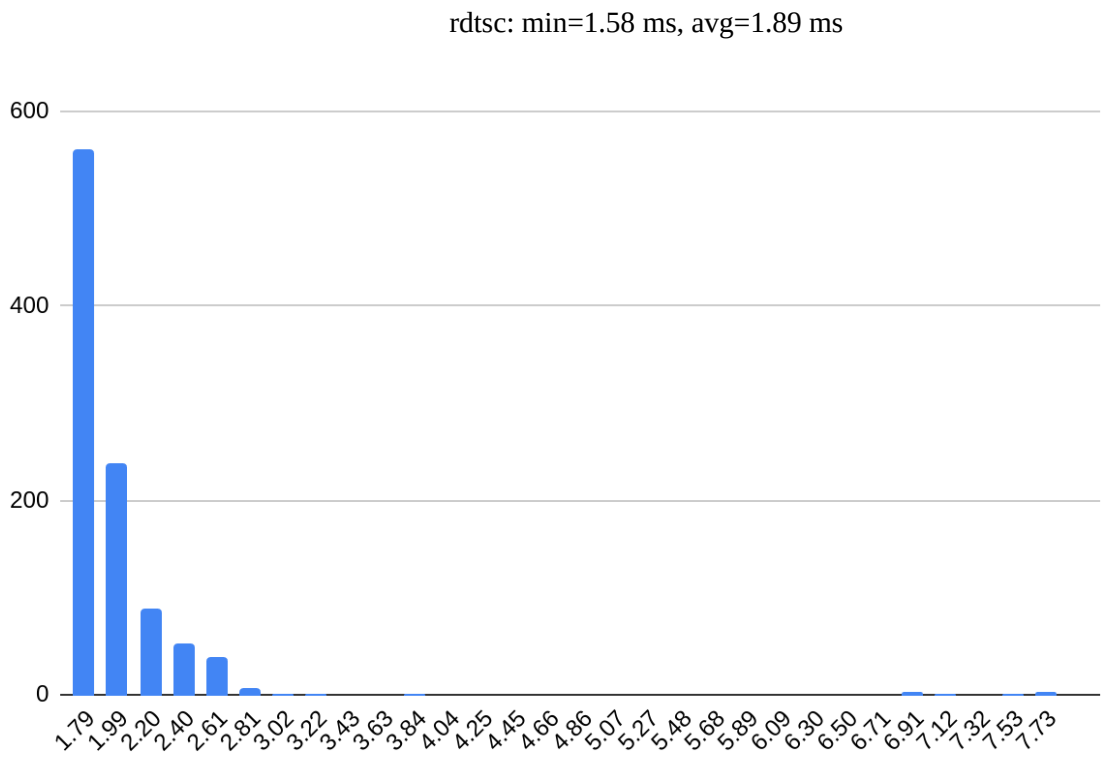
Все измерения проводились с N=2048*32. Для таблиц использовалось K=5, для гистограмм — брали K=1000. Модель процессора: Intel(R) Core(TM) i5-10300H CPU @ 2.50GHz

clock_gettime: min=1.59 ms, avg=1.89 ms



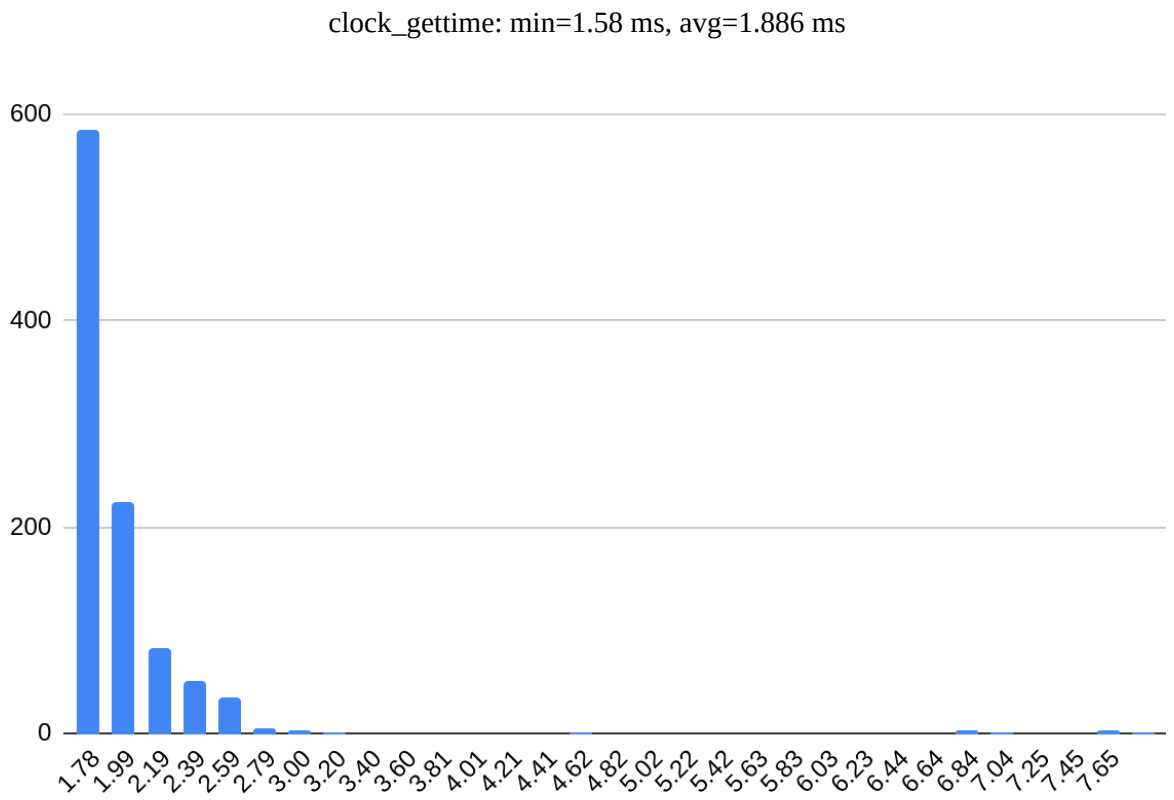
№ замера	1	2	3	4	5
Intel Core i5 (округляем, мс)	7.703813	7.584663	7.448459	7.712836	7.716593

СКО: 0.58 мс. Доверительный интервал при p=95: (1.86 мс; 1.93 мс)



№ замера	1	2	3	4	5
Intel Core i5, нс	7.452062	7.731094	7.714036	7.653818	7.591606

СКО: 0.569 мс. Доверительный интервал при p=95: (1.859 мс; 1.93 мс)



№ замера	1	2	3	4	5
Intel Core i5, нс	7.5834576	7.5359184	7.553696	7.5405792	7.6506656

СКО: 0.5737 мс. Доверительный интервал при p=95: (1.8507 мс; 1.9219 мс)

Выводы

В соответствии с выбранным заданием произведены измерение времени выполнения заданного фрагмента программного кода с использованием стандартных методов и обращения к таймеру внутри ядра системы. Каждый замер повторялся не менее K=1000 раз, гистограммы подтверждают в наличии погрешности измерения.